

CLASE M6 - LECCIÓN 1: INTRODUCCIÓN A NODE.JS.

OBJETIVO DE LA CLASE

Comprender qué es Node.js, para qué se usa, qué rol cumple en el backend, cómo se diferencia del JavaScript del navegador, cómo se usa npm y cómo levantar un primer servidor básico con Node puro.

1. INICIO DE CLASE - ACTIVACIÓN

Preguntas generadoras

Antes de explicar Node, partiría con estas preguntas:

Hasta ahora hemos usado JavaScript principalmente en el navegador.

¿Qué cosas podíamos hacer con JavaScript en el frontend?

- Manipular el DOM.
- Escuchar eventos.
- Validar formularios.
- Consumir APIs con fetch.
- Cambiar estilos o contenido dinámicamente.

Ahora pensemos como equipo de desarrollo: si una empresa necesita guardar datos, responder solicitudes, validar usuarios o entregar información a muchas personas, ¿basta solo con HTML, CSS y JS en el navegador?

No. Necesitamos backend/servidor.

Analicemos:

Una pequeña empresa necesita crear una aplicación web donde los usuarios puedan consultar información desde el navegador. Para eso necesitamos un servidor que reciba una petición y entregue una respuesta.

2. PRESENTACIÓN DEL MÓDULO Y NODE.JS

Node.js nos permite usar JavaScript fuera del navegador. Eso significa que el mismo lenguaje que ya venimos trabajando ahora lo podemos ejecutar en el computador como una aplicación independiente.

JavaScript antes:

Navegador -> HTML/CSS/JS -> interacción visual

JavaScript con Node:

Servidor -> Node.js -> procesa solicitudes y entrega respuestas

Analiza y responde:

¿Qué diferencia habría entre hacer clic en un botón con JavaScript en el navegador y pedir datos a un servidor con Node?

En el navegador el código responde a acciones del usuario. En el servidor, Node responde a solicitudes HTTP, lee datos, procesa lógica y devuelve respuestas.

3. ¿QUÉ ES NODE.JS?

Analicemos:

La idea más importante aquí es que Node.js toma JavaScript y lo lleva al lado del servidor. Ya no dependemos solo del navegador para ejecutar código JavaScript.

Node.js se construye sobre el motor V8 de Google Chrome. V8 es el motor que interpreta JavaScript. Node toma esa capacidad y la usa fuera del navegador.

Node.js es un **entorno de ejecución**.

No es:

- Un lenguaje nuevo.
- Un framework.
- Una base de datos.

Sí es:

- Una plataforma para ejecutar JavaScript fuera del navegador.
- Una herramienta para crear servidores.
- Una base para frameworks como Express.

Mini ejemplo conceptual

```
console.log("Hola desde Node.js");
```

En el navegador usaríamos la consola del navegador.

En Node lo ejecutamos desde la terminal:

```
node app.js
```

4. FRONTEND, BACKEND Y MIDDLEWARE

La diapositiva muestra una relación entre **frontend**, **backend**, **servidor**, **base de datos** y **middleware**.

Hasta ahora ustedes han estado trabajando principalmente en el frontend. El frontend es lo que el usuario ve y usa: botones, formularios, tarjetas, navegación. Pero cuando el usuario necesita guardar o consultar información, debe comunicarse con un backend.

Node.js aparece como una tecnología para construir ese backend.

Observa:

Usuario => Frontend => Request => Backend Node => Response => Frontend

Cliente en el restaurante => Garzon => Cocina => Garzon = Cliente

CLiente => Usuario

Garzon = Frontend

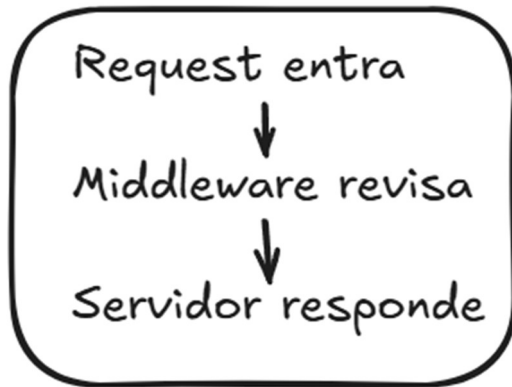
Cocina = Backend

Pedido = Request

Plato servido = Response

INTRODUCCIÓN TEMPRANA A MIDDLEWARE

Middleware es una función que se ejecuta entre la petición y la respuesta. Sirve para validar, registrar, transformar o controlar lo que pasa antes de responder.



Todavía no es necesario profundizar mucho, solo dejarlo instalado como concepto.

5. ¿QUIÉNES USAN NODE.JS?

La presentación muestra ejemplos como PayPal, Walmart, Netflix, LinkedIn, NASA y Uber.

Esto nos ayuda a entender que Node.js no es una herramienta de juguete. Se usa en empresas grandes porque permite crear aplicaciones rápidas, escalables y orientadas a muchas peticiones simultáneas.

¿Por qué creen que empresas como Netflix o Uber podrían necesitar tecnologías capaces de manejar muchas solicitudes al mismo tiempo?

Respuestas esperadas:

- Muchos usuarios conectados.
- Peticiones simultáneas.
- Necesidad de respuestas rápidas.
- Servicios distribuidos.
- APIs internas y externas.

En la industria, Node se usa mucho para crear APIs, servicios backend, herramientas internas, automatizaciones y aplicaciones web completas. En este bootcamp lo usaremos como base para entender cómo trabaja un backend JavaScript.

6. NPM: NODE PACKAGE MANAGER

Comentario sugerido

Cuando trabajamos en proyectos reales, no escribimos todo desde cero. Usamos librerías, paquetes y herramientas desarrolladas por la comunidad. npm es el administrador de paquetes que viene con Node.

npm permite:

- Crear proyectos Node.
- Instalar paquetes.
- Quitar paquetes.
- Actualizar dependencias.
- Ejecutar scripts.
- Compartir proyectos.

Conceptos clave

Ejemplo guiado 1 - Crear proyecto Node

npm: Administrador de paquetes

package.json: ficha del proyecto

node_modules: carpeta donde se descargan paquetes

dependencies: Paquetes que necesita la app para funcionar

devDependencies: paquetes usados durante el desarrollo

Crear carpeta:

```
mkdir clase-node-01
```

```
cd clase-node-01
```

Inicializar proyecto:

```
npm init -y
```

Explicar:

El comando npm init -y crea automáticamente un archivo package.json con configuración básica.

7. EJERCICIO PRÁCTICO 1 - PRIMER PROGRAMA CON NODE

Objetivo

Comprobar que Node ejecuta JavaScript fuera del navegador.

Paso 1: crear archivo

Crear app.js.

```
console.log("Hola desde Node.js");  
console.log("Este código no se está ejecutando en el navegador");
```

Paso 2: ejecutar

node app.js

Resultado esperado

Hola desde Node.js

Este código no se está ejecutando en el navegador

Analicemos:

¿Dónde se mostró el resultado?

¿En el navegador o en la terminal?

Entonces, ¿qué nos permite hacer Node?

Ejecutar JavaScript fuera del navegador.

8. COMANDOS NPM

La presentación muestra comandos como:

npm install nombre_paquete

npm install jest

npm install jest@23.6.0

npm uninstall jest

npm update

npm -v

npx nombre_paquete

Estos comandos son parte del día a día de un desarrollador backend o full stack. No se trata de memorizarlos todos hoy, sino de entender qué problema resuelve cada uno.

Explicación clara

```
npm install express
```

Instala un paquete.

```
npm uninstall express
```

Elimina un paquete.

```
npm -v
```

Muestra la versión de npm.

```
npx paquete
```

Ejecuta un paquete sin instalarlo necesariamente de forma permanente.

9. INSTALACIÓN DE NODE.JS

En esta parte debemos asegurarnos de que todos tengan Node instalado. Si alguien no lo tiene funcionando, no seguimos como si nada: lo resolvemos porque es la base del módulo.

Checklist de instalación

Pedir que ejecuten:

```
node -v
```

Luego:

```
npm -v
```

Resultado esperado

Algo similar a:

```
v20.x.x
```

```
10.x.x
```

No importa que no todos tengan exactamente la misma versión, pero sí que ambos comandos respondan.

Errores comunes

Error 1: “node no se reconoce como comando”

Posibles causas:

- Node no está instalado.
- La terminal estaba abierta antes de instalar.

- Node no quedó agregado al PATH.

Solución:

Cerrar y abrir terminal. Si sigue fallando, reinstalar Node marcando la opción de agregar a PATH.

Error 2: npm falla al instalar paquetes

Posibles causas:

- Mala conexión.
- Permisos.
- Carpeta con caracteres extraños.
- Instalación incompleta.

Solución inicial:

npm cache verify

Luego probar nuevamente.

10. HOLA MUNDO EN NODE CON SERVIDOR HTTP

La presentación muestra un primer servidor usando el módulo http, createServer, res.writeHead, res.end y server.listen(8000).

Comentario sugerido

“Ahora viene el primer gran salto: ya no solo ejecutaremos un console.log, sino que levantaremos un servidor. Eso significa que nuestro computador escuchará peticiones en un puerto y responderá algo.”

Código explicado línea por línea

Crear archivo servidor.js:

```
'use strict';

// Importamos el módulo http que viene incluido en Node.js.
// No necesitamos instalarlo con npm porque es un módulo nativo.
const http = require('http');

// Creamos el servidor.
// Esta función se ejecuta cada vez que llega una petición al servidor.
const server = http.createServer(function (req, res) {

  // Definimos el código de estado 200, que significa "OK".
  // También indicamos que la respuesta será texto plano.
```



```
res.writeHead(200, { 'content-type': 'text/plain' });

// Enviamos la respuesta final al navegador.
res.end('Hola Mundo');
});

// El servidor queda escuchando en el puerto 8000.
server.listen(8000);

// Mensaje para confirmar en terminal que el servidor está activo.
console.log('Servidor escuchando en http://localhost:8000');
```

Ejecutar

`node servidor.js`

Probar en navegador

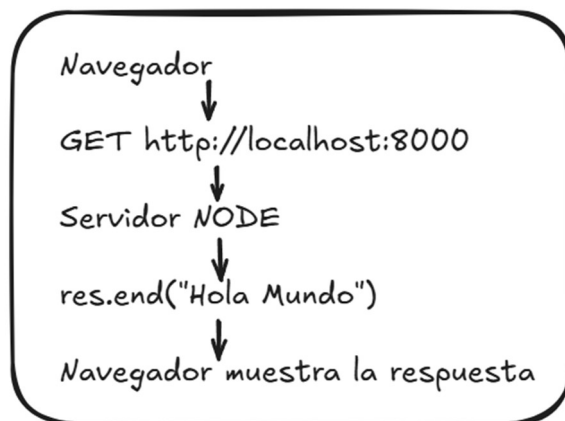
Abrir:

`http://localhost:8000`

Debe aparecer:

`Hola Mundo`

11. EXPLICACIÓN DEL FLUJO REQUEST/RESPONSE



¿Quién hace la petición?

El navegador.

¿Quién responde?

El servidor Node.

¿Qué significa localhost?

Es mi propio computador actuando como servidor.

¿Qué significa el puerto 8000?

Es una puerta lógica por donde el servidor escucha solicitudes.

12. EJERCICIO PRÁCTICO 3 - PERSONALIZAR LA RESPUESTA

Consigna

Modificar el servidor para que responda con un mensaje personalizado.

Código base

```
'use strict';

const http = require('http');

const server = http.createServer(function (req, res) {
  res.writeHead(200, { 'content-type': 'text/plain' });
  res.end('Bienvenidos al módulo 6 - Node JS');
});

server.listen(8000);

console.log('Servidor escuchando en http://localhost:8000');
```

Puedes cambiar el mensaje por:

Mi primer servidor backend con JavaScript

Luego reiniciar el servidor.

Punto importante

Con Node puro, cada vez que cambiamos el archivo debemos detener y volver a levantar el servidor manualmente.

Para detener:

Ctrl + C

Para levantar:

node servidor.js

Esto conecta más adelante con Nodemon.

13. EJERCICIO PRÁCTICO 4 - RESPONDER SEGÚN LA RUTA

Aunque todavía no han visto Express, este ejercicio ayuda a entender por qué Express será útil.

Consigna

Crear un servidor Node puro que responda distinto según la URL.

Código

```
'use strict';

const http = require('http');

const server = http.createServer(function (req, res) {
  res.writeHead(200, { 'content-type': 'text/plain; charset=utf-8' });

  if (req.url === '/') {
    res.end('Página de inicio');
  } else if (req.url === '/productos') {
    res.end('Listado de productos');
  } else if (req.url === '/contacto') {
    res.end('Página de contacto');
  } else {
    res.writeHead(404, { 'content-type': 'text/plain; charset=utf-8' });
    res.end('Página no encontrada');
  }
});

server.listen(8000);

console.log('Servidor escuchando en http://localhost:8000');
```

Probar

http://localhost:8000

http://localhost:8000/productos

http://localhost:8000/contacto

http://localhost:8000/cualquier-cosa

Esto funciona, pero imaginen una aplicación con 50 rutas. El código crecería mucho y sería difícil de mantener. Por eso en las próximas clases usaremos Express, que simplifica la creación de rutas y servidores.

14. VENTAJAS DE NODE.JS

Comentario sugerido

“Node destaca porque trabaja muy bien con operaciones de entrada y salida: leer archivos, recibir peticiones, consultar servicios externos, responder a usuarios. Su modelo basado en eventos permite manejar muchas solicitudes sin bloquear todo el programa.”

Explicación simple

Node es útil para:

- Crear servidores web.
- Crear APIs REST.
- Consumir APIs.
- Automatizar tareas.
- Leer y escribir archivos.
- Trabajar con bases de datos.
- Crear aplicaciones en tiempo real.

Node usa un modelo no bloqueante. Eso significa que puede iniciar una tarea, seguir atendiendo otras cosas y luego volver cuando la tarea termina.

Mini ejemplo:

```
console.log("Inicio");

setTimeout(() => {
  console.log("Tarea asincrónica terminada");
}, 2000);

console.log("Fin");
```

Resultado:

Inicio

Fin

Tarea asincrónica terminada

Pregunta de reflexión

¿Por qué creen que esto puede ser útil en un servidor?

Respuesta esperada:

Porque un servidor no puede quedarse detenido esperando una sola tarea. Debe seguir atendiendo más solicitudes.

16. CIERRE DE CLASE

Preguntas finales

- ¿Qué es Node.js?
- ¿Qué diferencia hay entre JavaScript en navegador y JavaScript en servidor?
- ¿Qué es npm?
- ¿Qué comando usamos para ejecutar un archivo Node?
- ¿Qué hace `server.listen(8000)`?
- ¿Qué problema aparece si tenemos muchas rutas usando solo Node puro?

Hoy dimos el primer paso hacia el backend. Aprendimos que Node.js nos permite ejecutar JavaScript fuera del navegador, que npm nos ayuda a gestionar paquetes y que con el módulo `http` podemos crear un servidor básico. La próxima conexión natural será Express, porque nos permitirá construir servidores de forma más ordenada, con rutas, middlewares y estructura profesional.

Tarea sugerida

Crear un repositorio llamado:

`m6-clase1-node-servidor`

Debe contener:

`app.js`

`package.json`

`README.md`

El servidor debe responder al menos a 4 rutas y manejar error 404.

README mínimo

`# Primer servidor con Node.js`

`Este proyecto corresponde a la primera práctica del Módulo 6.`

Comandos

Instalar dependencias:

```
npm install
```

Ejecutar servidor:

```
node app.js
```

Rutas disponibles

```
- /
```

```
- /nosotros
```

```
- /servicios
```

```
- /contacto
```